

Jour 2

Le bug Ariane 5 → module pour mesurer l'altitude d'une fusée sauf que mauvaise variable utilisé → cout perdu a cause de une ligne de code → 500 millions

3 facons de déclaré une variable :

var → n'importe où dans le code , avec inférence de type

Déclaire uniquement dans les fonctions :

- test := "test"
- x,y := 10, 20

Echange de variable élégant :

- x, y = y, x

Variable avec plusieurs déclaire groupé :

```
var (  
    serveur string = 'localhost'  
    port int = 8888  
    actif bool = true  
)
```

Pour les package utilisé var pas :=

iota → Générateur d'énumérations, elle s'incrémente à chaque constante dans un group const()

```
ex :  
const (  
    DEBUG NiveauLog = iota //0  
    INFO //1  
    WARNING //2  
    ERROR //3  
    CRITICAL //4  
)
```

iota avec calcul - taille de stockage

```
const (  
    octet = 1  
    Ko = 1 << (10 * (iota +1)) // 1024  
    etc ...  
)
```

Une fonction en Go ne peut pas être surchargée

Retour multiple :

```
if b == 0 {  
    return 0, fmt.Errorf("division par zéro : impossible de diviser %v par zéro, a")  
}
```

avec _ Ignorer une valeur de return

Les erreurs Go sont plus explicites que les exceptions de Java

Fonctions variadic :

En Go, une fonction variadic est une fonction qui peut accepter un nombre variable d'arguments d'un même type.

Fonctions closure :

Une fonction qui se souvient de l'environnement dans lequel elle a été créée

C'est une fonction qui return une fonction

avantage : optimisation de l'utilisation de ressource

utile pour des callback, HTTP ou tris personnalisés

La seule boucle en go est le for

exemple while en go

```
n:=1  
for n < 100 {  
    n*=2  
}
```

fallthrough est un mot-clé utilisé dans une instruction switch pour forcer l'exécution du bloc suivant, même si sa condition ne correspond pas.

un slice est une structure très importante : c'est une vue flexible sur un tableau. N'a pas de longueur fixe.

ex :

```
s := []int{1, 2, 3}
```

make : sert à créer et initialiser des slices.

len() : donne le nombre d'éléments dans un slice.

cap() : donne la capacité maximale du slice (espace disponible en mémoire).

append() : sert à ajouter un ou plusieurs éléments à un slice.

Maps : dictionnaire cle/valeur

on accede a une donné d'un tableau pas via l'index mais via la valeur recherché en utilisant **maps**

Déf d'un type struct

```
type Produit struct {  
    Nom string  
    Prix float64  
    Categories []string  
}
```

```
p1 := Produit("Iphone", 17.00, []string{'High-tech'})
```

Recommande :

```
p1 := Produit(  
    "Iphone",  
    17.00,  
    []string{'High-tech'}  
)
```

Pointeur (pointer) : une variable qui contient l'adresse mémoire d'une autre variable. Il permet de modifier directement la valeur originale.

Receveur (receiver) : en Go, c'est la variable associée à une fonction pour un type (dans une méthode). Il sert à donner un comportement à une structure.

Struct Tags — métadonnées pour JSON et plus

Les tags sont des chaînes entre backticks après chaque champ :

```

type Utilisateur struct {
    ID    int    `json:"id"          // sérialisé comme "id"
    Prenom string `json:"prenom"`
    Email  string `json:"email,omitempty"` // omis si vide ("" )
    Age    int    `json:"age,omitempty"` // omis si 0
    Interne string `json:"-"`          // JAMAIS sérialisé
    MDP    string `json:"-"`          // mot de passe : jamais !
}

```

/Exemple avec JSON (anticipation Jour 3) :

```

import "encoding/json"

u := Utilisateur{ID: 1, Prenom: "Alice", Email: "alice@go.dev", Age: 30}
data, _ := json.Marshal(u)
fmt.Println(string(data))
// {"id":1,"prenom":"Alice","email":"alice@go.dev","age":30}

// Unmarshal : JSON -> struct
jsonStr := `{"id":2,"prenom":"Bob"}`
var u2 Utilisateur
json.Unmarshal([]byte(jsonStr), &u2)
fmt.Println(u2.Prenom) // Bob

```

Pour fonction des struct si pas de majuscule alors equivaut au private si majuscule dispo n'importe ou

Nommage de fichier Go sans -_ etc et en camelCase

defer – garantir l'exécution d'une fonction

Le defer permet de retarder l'exécution d'une fonction jusqu'à ce que la fonction parente se termine.

Package standard incontournable

```

strings : manipulation de chaines
strings.Contains("hello go", "go") // true
strings.ToUpper("hello")           // "HELLO"
strings.Split("a,b,c", ",")        // ["a","b","c"]

```

```
strings.TrimSpace(" hello ")    // "hello"  
strings.HasPrefix("golang", "go") // true
```

```
strconv : conversions string ↔ types  
strconv.Itoa(42)           // "42"  
strconv.Atoi("123")        // 123, nil  
strconv.ParseFloat("3.14", 64) // 3.14, nil
```

```
math : fonctions mathématiques  
math.Abs(-5.0)             // 5.0  
math.Sqrt(16)              // 4.0  
math.Pow(2, 10)            // 1024.0  
math.Max(3.0, 5.0)         // 5.0
```

```
sort : tri  
nums := []int{3, 1, 4, 1, 5, 9}  
sort.Ints(nums)            // tri sur place  
strs := []string{"b", "a", "c"}  
sort.Strings(strs)
```

```
time : date et heure  
now := time.Now()  
fmt.Println(now.Format("02/01/2006 15:04:05"))  
duration := 5 * time.Second  
time.Sleep(duration)
```